

ADS Lab Exam – PGCP-AC August 2026

C-DAC Mumbai · Module End Lab Exam · Total Marks (40)

Question 1: Queue using two stacks

Class name: QueueUsingTwoStacks

Problem statement

Implement a FIFO queue of integers using only two stacks: stack A (inStack) and stack B (outStack).

- enqueue(x): push x onto stack A.
- dequeue(): if stack B is empty, first pop every element from A and push it onto B; then pop from B and return it.
- peek(): same as dequeue, but returns the front element without removing it.

Rules

- Use only stack operations (push, pop, peek, isEmpty). You may use java.util.ArrayDeque as a stack or write your own stack class. Do not use Queue, LinkedList or any queue method (offer, poll, addLast, ...).
- If dequeue or peek is called on an empty queue, print EMPTY (do not crash).

Suggested structure

```
public class QueueUsingTwoStacks {  
    // two stacks only  
    public void enqueue(int x) { }  
    public int dequeue()      { } // EMPTY case handled by caller or exception  
    public int peek()         { }  
    public int size()         { }  
    public static void main(String[] args) { /* read operations, print results */ }  
}
```

Input format

First line: integer Q, the number of operations.

Next Q lines: one operation each:

- E x: enqueue integer x
- D: dequeue and print the removed value
- P: print the front value without removing it
- S: print the number of elements in the queue

Output format

For every D, P and S operation print one line: the value, or EMPTY when the queue has no elements (for D and P). E prints nothing.

Sample test cases

Sample 1 – Input

```
7
E 10
E 20
E 30
D
E 40
D
P
```

Sample 1 – Expected output

```
10
20
30
```

Sample 2 – Input

```
5
D
E 5
P
D
D
```

Sample 2 – Expected output

```
EMPTY
5
5
EMPTY
```

Question 2: Find duplicates in a linked list

Class name: LinkedListDuplicates

Problem statement

Build a singly linked list from the input values. Find every value that occurs more than once in the list and print it together with the number of times it occurs.

Rules

- Write your own Node class.
- Do not copy the values into an array, ArrayList, HashSet, HashMap or any other collection. Work only on the linked list.

- Print the duplicated values in the order in which they first appear in the list. Each duplicated value must be printed only once.

Suggested structure

```
static class Node { int data; Node next; Node(int d) { data = d; } }
static void printDuplicates(Node head) { } // traverse the list and count
occurrences
```

Input format

First line: integer n.

Second line: n integers (may be negative), separated by spaces (absent when n = 0).

Output format

Either the single line No duplicates, or the line Duplicates: followed by one line per duplicated value: value (k times).

Sample test cases

Sample 1 – Input

```
8
4 2 7 2 9 4 4 1
```

Sample 1 – Expected output

```
Duplicates:
4 (3 times)
2 (2 times)
```

Sample 2 – Input

```
5
10 30 20 50 40
```

Sample 2 – Expected output

```
No duplicates
```

Question 3: Binary Search Tree – insert and inorder traversal

Class name: BSTInorder

Problem statement

Insert the given keys one by one, in the given order, into an initially empty Binary Search Tree (BST). Then print the keys using an inorder traversal of the tree.

Rules

- Write your own Node class (int key; Node left, right;) and an insert method (recursive or iterative). Do not use TreeMap, TreeSet or any other collection to store the keys.
- Smaller keys go to the left, bigger keys go to the right. A key that is already present is ignored.
- The keys must be printed by an inorder traversal (left, node, right) of the tree, not by sorting them.

Suggested structure

```
static class Node { int key; Node left, right; Node(int k) { key = k; } }  
static Node insert(Node root, int key) { }  
static void inorder(Node root) { }
```

Input format

First line: integer n ($0 \leq n \leq 1000$).

Second line: n integers (may be negative), the keys in insertion order (absent when n = 0).

Output format

One line: Inorder: followed by the keys separated by single spaces. If no key was inserted, print Inorder: Empty.

Sample test cases

Sample 1 – Input

```
7  
50 30 70 20 40 60 80
```

Sample 1 – Expected output

```
Inorder: 20 30 40 50 60 70 80
```

Sample 2 – Input

```
8  
15 10 20 10 25 5 15 30
```

Sample 2 – Expected output

```
Inorder: 5 10 15 20 25 30
```